

## Table of Contents

|                                                            |   |
|------------------------------------------------------------|---|
| Handling the Recipes redirect from the Home Page.....      | 1 |
| my-data.ts implementation – using BehaviorSubject.....     | 1 |
| Reference List for research behind BehaviorSubject: .....  | 2 |
| A third service – storage.ts .....                         | 2 |
| Adding a third icon and solidifying header bar style ..... | 3 |
| Adding the Angular number pipe .....                       | 3 |
| Reference List for Pipes .....                             | 3 |
| Ion Spinner for loading .....                              | 4 |
| Reference List for Ion Spinner .....                       | 4 |
| Optional Chaining – code robustness.....                   | 4 |
| Reference List for Optional Chaining .....                 | 4 |
| Conditional (ternary) operator – is it a favourite? .....  | 5 |
| Reference List for Conditional (ternary) operator .....    | 5 |

## Handling the Recipes redirect from the Home Page

To simplify the redirect to the Recipes page and make it dynamic rather than to a static Recipes page, I changed the app.routes.ts file routing to the Recipes page by specifying the path as recipes/:id. This allowed for a different approach when developing the Recipes page.

```
{  
  path: 'recipes/:id',  
  loadComponent: () => import('./recipes/recipes.page').then( m =>  
m.RecipesPage)  
},
```

## my-data.ts implementation – using BehaviorSubject

Research led me to choose to use BehaviorSubject, which goes beyond what was taught during the course. BehaviorSubject “allows to push and pull values to the underlying Observable, [...] represents a value that changes over time, [...] every subscriber will always

*get the initial or the last value that the subject emits”* (Patel, 2018). I felt this would be efficient for our purpose of Search Results, as well as defining Favourites and Measurement units which can change throughout app usage and be re-defined by the user.

### Reference List for research behind BehaviorSubject:

Morony, Josh, *Using BehaviorSubject to Handle Asynchronous Loading in Ionic*. Published on May 15<sup>th</sup>, 2021. Accessed on December 7<sup>th</sup>, 2025. Available at <https://eliteionic.com/tutorials/using-behaviorsubject-to-handle-asynchronous-loading-in-ionic/#:~:text=A%20BehaviorSubject%20is%20a%20type,different%20to%20a%20standard%20observable.>

Bacancy (Website), *What is the Difference Between BehaviorSubject and Observable?*. Published on May 9<sup>th</sup>, 2024. Accessed on December 7<sup>th</sup>, 2025. Available at <https://www.bacancytechnology.com/qanda/angular/difference-between-behaviorsubject-and-observable>

Hashim KM, *Behavior Subject in Angular — Implementation*. Published on January 29<sup>th</sup>, 2024. Accessed on December 7<sup>th</sup>, 2025. Available at <https://medium.com/@hashimk155/behaviour-subject-in-angular-implementation-b16d7d4c85eb>

Patel, Parth, *Benefits of using BehaviorSubject in Angular*. Published on March 26<sup>th</sup>, 2018. Accessed on December 7<sup>th</sup>, 2025. Available at <https://www.cmarix.com/blog/benefits-of-using-behaviorsubject-in-angular/>

### A third service – storage.ts

Choosing to create a third service, storage.ts, along with my-http.ts and my-data.ts, is a decision based on the Single Responsibility Principle (SRP) in software architecture, which was taught to us in a different module, and whose concepts I believe also apply to Ionic Apps, not just Java-developed code.

The Storage service’s single responsibility is to handle persistence, meaning it is only responsible for the saving and retrieving of data from the device's local storage using asynchronous operations (Promises) as taught in this module.

## Adding a third icon and solidifying header bar style

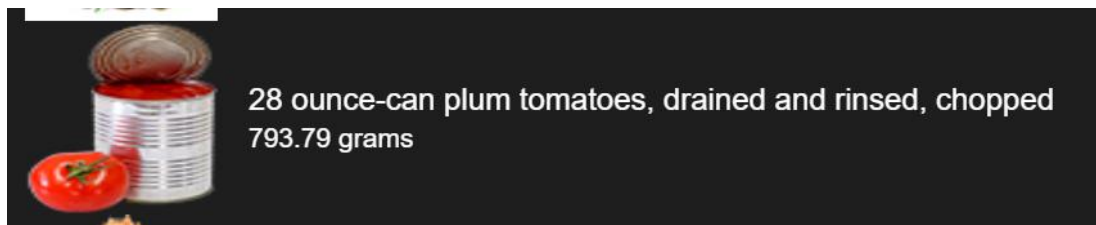
I added a third icon in the header bar across all pages – a home icon. This allows the user to easily jump back to the home page, without having to always click the back button.

Furthermore, I've applied the color: primary value to the header across all the pages of the app, to solidify the appearance.

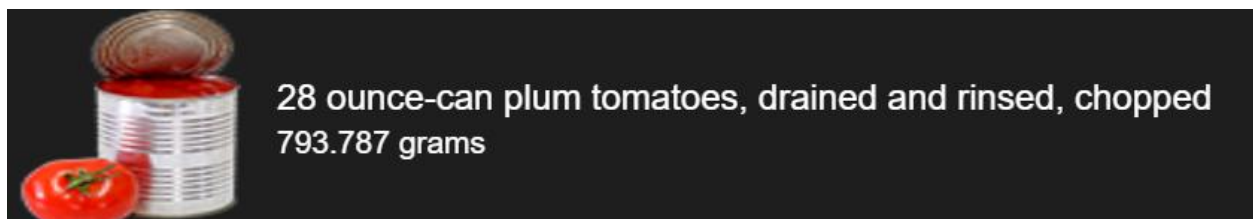
## Adding the Angular number pipe

In `recipes.page.html`, you will see that I have added the pipe to transform the way the measurement units are presented. This is to ensure that amounts will be rounded up nicely. I chose the format `| number:'1.0-2'`, which basically means to show the number with at least one integer digit (1.), and between 0 and 2 decimal places (0-2).

As an example, here is a measurement with pipe implemented:



Here is the same measurement without pipe implemented:



## Reference List for Pipes

Angular Dev, *Pipes*. Accessed on December 14<sup>th</sup>, 2025. Available at <https://angular.dev/tutorials/learn-angular/22-pipes>

## Ion Spinner for loading

To improve the app's loading, in case things might take longer to load than expected, I added in the Ion-Spinner on same pages, such as `recipes.page.html`

```
<div *ngIf="isLoading">
  <ion-spinner name="crescent"></ion-spinner>
  <p>Loading recipe...</p>
</div>
```

## Reference List for Ion Spinner

Angular Dev, *ion-spinner*. Accessed on December 14<sup>th</sup>, 2025. Available at <https://ionicframework.com/docs/api/spinner>

## Optional Chaining – code robustness

Implemented optional chaining in `recipes.page.html` in case that the recipe details title became missing or didn't load. In this scenario, rather than the app crashing, the returning result would be "undefined". It is also known as the "safe navigator operator" (Zayani, 2023).

```
<ion-title>{{ recipeDetails?.title || 'Recipe Details' }}</ion-title>
```

## Reference List for Optional Chaining

Zayani, Zied, *Safe Navigation Operator – Optional Chaining (JS and Angular)*. Published on February 20<sup>th</sup>, 2023. Accessed on December 14<sup>th</sup>, 2025. Available at <https://medium.com/@zayani.zied/safe-navigation-operator-optional-chaining-js-and-angular-d253431a2625>

MDN Mozilla, *Optional chaining (?)*. Accessed on December 14<sup>th</sup>, 2025. Available at [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Optional\\_chaining](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Optional_chaining)

## Conditional (ternary) operator – is it a favourite?

This was used to determine in the Recipes details page whether a recipe is a favourite or not, in the favourite toggle button:

```
<ion-button (click)="toggleFavorite()" [color]="isFavorite ? 'danger' :  
'medium'">  
  <ion-icon slot="start" name="heart"></ion-icon>  
  {{ isFavorite ? 'Remove from Favourites' : 'Add to Favourites' }}  
</ion-button>
```

It is a convenient way to write in an if-else statement inline. Its structure is a condition, followed by ?, and then if the condition is true, do x, if the condition is false, do y. For example, in my case, if the recipe is favourite, then the colour of the button should be “danger” (red), if false (recipe is not yet marked as favourite), then the colour of the button is “medium” (a light grey). Same applies for the button itself – if the recipe is marked as a favourite, then show the option to “Remove from Favourites”. If it’s false – as in, the recipe is not marked as a favourite -then you have the option to “Add to Favourites”.

## Reference List for Conditional (ternary) operator

MDN Mozilla, Conditional (ternary) operator. (?.). Accessed on December 14<sup>th</sup>, 2025.  
Available at [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Conditional\\_operator](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Conditional_operator)